

agora eu entendi a confusão toda, no tutorial microsoft esta escrito: `https://localhost:{PORT}/weatherforecast`, a parte do port era o numero da porta que seria gerado, só que eu fiquei tentando usando o port, depois eu corriji usando a porta mas esqueci de colocar o weatherforecast.

Exatamente — você matou a charada 👍

O trecho da documentação:

```
https://localhost:{PORT}/weatherforecast
```

significa:

- `{PORT}` → você substitui pela porta real (ex: 5001)
- `/weatherforecast` → **faz parte da rota**, não pode remover

🧠 O erro que aconteceu

Você fez isso (sem perceber):

```
https://localhost:5001/
```

Mas o correto era:

```
https://localhost:5001/weatherforecast
```

👉 Resultado:

- `/` → não existia → 404 ❌
- `/weatherforecast` → endpoint da API → funciona ✅

🔍 Por que `/weatherforecast` existe?

Porque o controller padrão vem assim:

```
[ApiController]
[Route("[controller]")]
public class WeatherForecastController : ControllerBase
```

👉 `[controller]` vira automaticamente:

```
WeatherForecast
```

Então a rota final fica:

```
/weatherforecast
```



Dica que evita esse tipo de erro

Sempre pense assim:

```
URL = domínio + porta + rota
```

Exemplo real:

```
https://localhost:5001/weatherforecast
```

Se faltar a **rota**, não adianta nada a porta estar correta.



Próximo nível (pra você evoluir rápido)

Você pode criar sua própria rota:

```
[Route("api/produtos")]
```

E acessar:

```
https://localhost:5001/api/produtos
```



Resumo final

- Porta correta ✓
- HTTPS configurado ✓
- Certificado confiável ✓
- Faltava só a rota ✓

👉 Ou seja: agora seu ambiente está 100% funcionando

Se quiser, posso te ajudar a transformar esse projeto em algo mais real (tipo API de cadastro, login, ou integração com banco) — isso conecta bem com o que você já vinha estudando 👍